`ComplexGitSync` 0000.04

Documentation

Nicolas Flipo

August 19, 2026

Contents

Chapter 1

Getting Started

This chapter guides a new user through the Multi-repo Sync CLI. The documented entry point is `cgitsync`.

The primary lifecycle is:

1. `initialise(.cgs)` → clone all repositories → `READY`
2. `initialise(.gts)` → load or restore a saved snapshot → `READY`
3. `pull(.cgs|.gts)` → resynchronise an existing tree
4. `checkout` / `add` / `commit` / `push` / `tag` → tree-wide Git operations
5. `freeze` → write the next `.gts` snapshot and update the project-local `.lgr`

1.1 Prerequisites

- Python 3.11 or later.
- Git 2.30 or later available on `PATH`.
- Working Git authentication for every remote used by the workspace.

1.2 Installation

Install `ComplexGitSync` from source with the Pixi-managed environment:

```
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync --help
```

1.3 Create or Check a Project Spec

A `.cgs` file is the local authoring spec for the repository tree. It normally contains only a project name and repository identifiers:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
  "github:flipoyo/CGSih1",
]
```

Each identifier uses `provider:owner/repository`. ComplexGitSync runs `PARSE -> NORMALIZE -> VALIDATE` and supplies deterministic defaults: `main` branches, `ssh`, `nested_config = auto`, and the repository name as its path. A unique repository matching the project name is mounted at `..` Advanced tables are reserved for overrides.

Canonical providers are `github`, `gitlab`, `codeberg`, and `custom`. Their known hosts are `github.com`, `gitlab.com`, and `codeberg.org`; SSH and HTTPS remotes are generated automatically. Custom hosting requires an explicit `gitprovider_url` and never guesses a host. For example, `codeberg:GX4G/GX4G` identifies owner `GX4G` and repository `GX4G` on Codeberg.

Copy `examples/template.cgs` when starting a specification. It combines plain repository identifiers with one inline override. Developers can compare it with `examples/normalized_template.cgs`, which shows the complete canonical document after normalization and is not intended as the usual hand-authored form.

The common inline options are `default_branch`, `fallback_branch`, `access_protocol`, `relative_path`, and `nested_config`. A relative path is resolved from the parent repository. Nested config can be `auto` to load the sole root-level `*.cgs`, `disabled` to stop discovery, or a relative `.cgs` path to select an exact file inside the repository.

```
$ pixi run cgitsync validate examples/complexgitsync.cgs
$ pixi run cgitsync view-tree examples/complexgitsync.cgs
```

A valid declared tree prints a lifecycle line such as:

```
DECLARED ready=false complete=true
```

1.4 Initialise the Workspace

Use `initialise` as the canonical first command. From a `.cgs` file it clones the full repository tree, validates it, writes the runtime snapshot under `<workspace>/.cgitsync/state/`, and leaves the tree `READY`.

```
$ pixi run cgitsync initialise examples/complexgitsync.cgs
```

The same canonical project definition can be authored directly. The `-repo` option is repeatable:

```
$ pixi run cgitsync initialise --project CGSil1 \
    --repo gitlab:CGS_test/CGSil1 \
    --repo codeberg:GX4G/GX4G
```

Use either a source file or `-project` plus `-repo`, never both. To serialize the CLI definition without initializing a workspace, use `create-cgs -project ... -repo ... -output FILE`.

When `-output-path` is omitted, `cgitsync` uses the workspace-level default `CGSPATH=../..`, then derives `CGSHOME=CGSPATH/<project-name>`. The explicit equivalent is:

```
$ pixi run cgitsync initialise examples/complexgitsync.cgs --output-path
    "$CGSPATH"
```

From a previously generated snapshot:

```
$ pixi run cgitsync initialise .cgitsync/state/complexgitsync.gts
```

Successful `.cgs` initialisation prints a readable operation sequence `GT-LOAD->GT-DISCOVER->GT-VALIDATE->` the explicit workflow `load->expand->validate->clone`, the per-repository `git clone` action, the resolved root path, and a compact tree outline. Structured log records also put the operation code first, for example `GT-DISCOVER`, `GT-VALIDATE`, `FS-PURGE`, or `GT-CLONE`. If a stale partial checkout or previous submodule link blocks the clone step, `initialise` fails explicitly and

prints `Try clean-init method`. Use `clean-init` to insert a purge phase between validation and cloning:

```
$ pixi run cgitsync clean-init examples/complexgitsync.cgs
```

The explicit operation sequence is `GT-LOAD->GT-DISCOVER->GT-VALIDATE->FS-PURGE->GT-CLONE`; the matching workflow is `load->expand->validate->purge->clone`. The purge phase can also be run on its own:

```
$ pixi run cgitsync purge examples/complexgitsync.cgs
```

`purge` removes generated clone state from `CGSHOME`: immediate child repository directories declared directly under the project root, project `*.lgr` files, and the root `.gitmodules` file.

1.5 Inspect the Runtime State

```
$ pixi run cgitsync view-tree .cgitsync/state/complexgitsync.gts
$ pixi run cgitsync view-tree .cgitsync/state/complexgitsync.gts --depth
    2 --collapse parent_2
$ pixi run cgitsync status
```

If a command accepts an optional snapshot and none is provided, `cgitsync` discovers the latest `.gts` from `CGSHOME/.cgitsync/state/`. `CGSHOME` can be set explicitly; for `initialise(.cgs)` the default is `CGSPATH=../..`, and later commands can also discover the workspace by walking upward from the current directory.

1.6 Run the Git Cycle

Once the tree is `READY`, the Multi-repo Sync commands operate on every repository in deterministic order:

```
$ pixi run cgitsync pull
$ pixi run cgitsync checkout feature/my-branch
$ pixi run cgitsync add
$ pixi run cgitsync commit "feat: update project CGS#1"
$ pixi run cgitsync push
$ pixi run cgitsync freeze-release release-2026.05 "release 2026.05"
$ pixi run cgitsync freeze release-2026.05
$ pixi run cgitsync launch-release release-2026.05
```

`pull` walks the tree parent-first, including the project root: `root -> parent -> leaf`. Mutation commands such as `add`, `commit`, `push`, and `freeze` walk the opposite direction: `leaf -> parent -> root`. When local files block the safe pull, the CLI suggests `cgitsync pull-force`; that recovery command is destructive and discards local uncommitted/untracked work before force-updating the same tree.

Pass `-gts <snapshot.gts>` when you want to pin a command to an explicit snapshot, and pass `-dry-run` to `add`, `commit`, `push`, or `freeze` to preview the plan without mutating repositories.

1.7 Log Files

Every CLI command writes a timestamped log file. On Linux the default location is `/.local/state/ComplexGi` if `XDG_STATE_HOME` is set, logs are written under that state directory instead. The CLI prints the resolved path as `log_file=<path>`.

1.8 Next Steps

See Chapter 4 for the command reference, document formats, preflight checks, and troubleshooting notes.

Chapter 2

Direct Python Object API

This chapter shows how to manipulate `ComplexGitSync` objects directly (without the CLI wrapper), including loading a `.gts` snapshot, reconstructing the runtime registry, mirroring identity into `GitTree`/`GitRepo`, and propagating a tag target.

For normal Multi-repo Sync usage, prefer the CLI lifecycle documented in the README and Getting Started guide. The reference test topology is CGSil1 (`https://gitlab.com/CGS_test/CGSil1`): `initialise -> pull -> checkout/add/commit/push -> freeze -> launch_release`.

The lower-level object/API lifecycle remains: `load(.cgs) -> .gts LOADED -> expand(.cgs|.gts) -> .gts PENDING -> validate(.cgs|.gts) -> .gts READY -> pull(.cgs|.gts) -> checkout(.gts) -> add -> git(tree,"commit",msg) -> git(tree,"push") -> git(tree,"tag",name) -> freeze`.

2.1 Programmatic Project Configuration

Project configuration does not require the CLI or an existing `.cgs` file. The public facade accepts authoring values non-interactively and delegates normalization, static validation, and optional serialization to `cgs_format.py`. It performs no Git or network operations.

```
from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
document = client.configure(
    "GX4G",
    ["codeberg:GX4G/GX4G"],
    output_path="GX4G.cgs",  # optional
)
reference_tree = document.to_git_tree()
```

The CLI commands `configure`, `create-cgs`, and direct `initialise -project/-repo` authoring are adapters over this method.

2.2 From Empty Registry to READY via `.gts`

```
from pathlib import Path

from ComplexGitSync import GitRepo, GitTree, RefKind, TreeLifecycleState
from ComplexGitSync.orchestre import GtsDocument,
   build_registry_from_gts_document
from ComplexGitSync.git_tree import WorkingGitTree

# 1) Empty registry lifecycle
empty_registry = WorkingGitTree()
```

```
assert empty_registry.recompute_tree_state() == TreeLifecycleState.
    UNLOADED

# 2) Load the CGSil1 runtime snapshot (.gts) and rebuild the dependency
    registry
cgshome = Path("../..")
gts_doc = GtsDocument.from_toml(cgshome / ".cgitsync/state/CGSil1.gts")
registry = build_registry_from_gts_document(gts_doc)
assert registry.lifecycle_state == TreeLifecycleState.READY

# 3) Rebuild GitTree/GitRepo identity objects from discovered registry
    entries
tree = GitTree()
for entry in registry.values():
    tree.add_repo(
        GitRepo(
            project_owner_name=entry.project_owner_name or "owner",
            project_name=entry.project_name or entry.name,
            gitprovider=entry.gitprovider,
            access_protocol=entry.access_protocol,
            commit_sha=entry.commit_sha,
        )
    )
```

2.3 Object-Level Tag Propagation

```
# Set the same tag target across all registry entries (parent -> leaves)
tree.propagate_tag(registry, "v1.2.3")
for entry in registry.values():
    assert entry.target_ref_kind == RefKind.TAG
    assert entry.target_ref_name == "v1.2.3"
```

This object-level flow is the same state model used by the CLI and `ComplexGitSyncClient`, but exposed directly for advanced automation and tests.

2.4 READY Methods in the Client API

The same lifecycle model is exposed by `ComplexGitSyncClient`. In practice, the workflow is:

- `load(...)` is the step-1 name for direct source loading.
- `expand(...)` is the canonical step-2 name; it runs nested `.cgs` discovery from parents to leaves (recursive).
- `validate(...)` is the canonical step-3 name; it accepts both `.cgs` and `.gts` sources.
- `initialise(...)` bootstraps a tree and must finish in `READY`; `clone(...)` is the direct clone entry point for a `.cgs` source; `pull(...)` resynchronises an existing tree.
- `git(tree, command, *args)` is the unified step-5 interface for `"commit"`, `"push"`, and `"tag"` operations. Each follows leaf-first ordering.
- `freeze(...)` emits the next persisted `.gts` state.

```
from pathlib import Path

from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
cgspath = Path("../..")
cgshome = cgspath / "CGSil1"

# 1. load the CGSil1 test-case .cgs -> .gts LOADED
client.load("../CGSil1.cgs")

# 2. expand(.cgs/.gts) -> .gts PENDING
print(client.expand("../CGSil1.cgs"))

# 3. validate(.cgs/.gts) -> .gts READY
client.validate("../CGSil1.cgs")

# 4. clone(.cgs/.gts)
# Same default as the CLI: output_path defaults to CGSPATH=../..
client.initialise("../CGSil1.cgs")

# Equivalent explicit form:
client.initialise("../CGSil1.cgs", output_path=cgspath)

# Recovery path for partial/stale clone state:
client.clean_initialise_cgs("../CGSil1.cgs", output_path=cgspath)

# Cleanup only:
client.purge_cgs("../CGSil1.cgs", output_path=cgspath)

# 5. READY methods - unified git interface
registry = client.get_dependency_registry()
# Pull uses ROOT -> PARENT -> LEAF, including the project root
   repository.
client.pull(".cgitsync/state/CGSil1.gts")
client.checkout("feature/my-branch")
# Mutations use LEAF -> PARENT -> ROOT.
client.add()
client.git(registry, "commit", "feat: update CGSil1 CGS#1")
client.git(registry, "push")
client.git(registry, "tag", "v1.2.3")

# 6. freeze -> next .gts id
client.freeze("release-2026.05")
```

Chapter 3

Architecture Overview

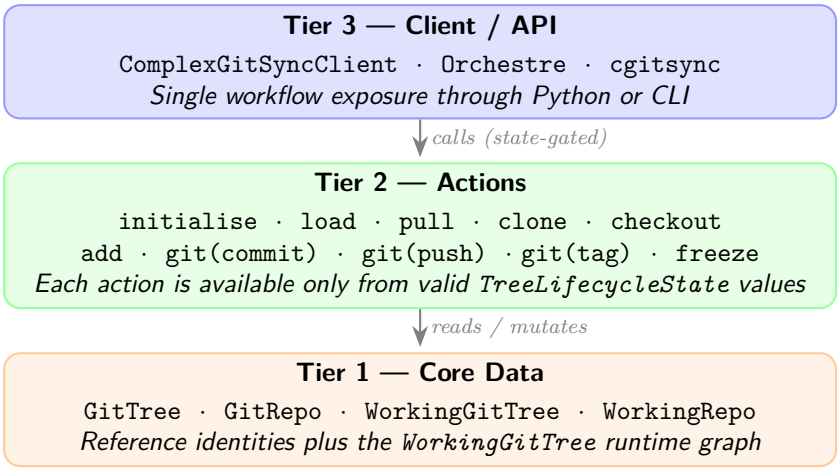
This chapter describes how `ComplexGitSync` is structured around three explicit tiers and explains the interaction between its classes. Related domain classes are grouped in modules according to the responsibility boundaries below.

3.1 Three-Tier Architecture

`ComplexGitSync` is organised into three tiers. Dependencies only flow **downward**: the Client calls Actions; Actions read and mutate Core Data; Core Data has no upward dependency.

Figure 3.1 shows the overall layout.

Figure 3.1: Three-tier architecture: Client/API exposes state-gated methods; Actions operate on Core Data; Core Data separates reference identities from the `WorkingGitTree` parent–leaf runtime graph.



3.2 Architectural Positioning

`ComplexGitSync` is positioned as a deterministic synchronisation layer *on top of Git*, not as a replacement for Git itself. It combines two distinct graph scopes:

- **Git DAG**: per-repository commit history and native remote semantics.
- **GitTree DAG**: workspace-level dependency graph coordinating parent/root/leaf propagation and ordering.

The system therefore differs from:

- plain Git usage (single repository scope),
- monorepo layouts (single storage boundary),
- submodule-only management (linking primitive without lifecycle orchestration),
- generic distributed sync engines (not constrained by Git-native '.gts'/'.lgr' deterministic contracts).

Figure 3.2 summarizes this positioning.

Figure 3.2: Architectural positioning matrix across Git scope and workspace coordination guarantees.

Approach	Primary scope	What it does not guarantee
Git (single repo)	Commit DAG per repository	Multi-repository propagation and coordinated lifecycle gating
Monorepo	One repository, one commit DAG	Independent repository identities and per-repo remote ownership
Submodule management	Parent repo references child revisions	Deterministic tree-wide mutation workflow and snapshot contracts
ComplexGitSync	Git DAG + GitTree DAG	Not a credential broker or remote policy authority

`freeze` materializes deterministic checkpoints as `.gts` snapshots (schema + SHA-256 hash), while `.lgr` assigns stable local ids and stores append-only ledger events for replayable local evolution.

Cross-declared nested repositories may initially resemble a local tangle at declaration time; runtime expansion normalizes this into a DAG via `fix_circularities`.

3.3 Tier 1 — Core Data

The reference model is centred on **`GitTree`**, which owns canonical **`GitRepo`** identities. Its runtime counterpart, **`WorkingGitTree`**, adds the parent–leaf graph of mutable **`WorkingRepo`** nodes. A project consists of one *root* repository that may nest *parent* repositories and terminal *leaf* repositories.

Figure 3.3 shows a representative project tree.

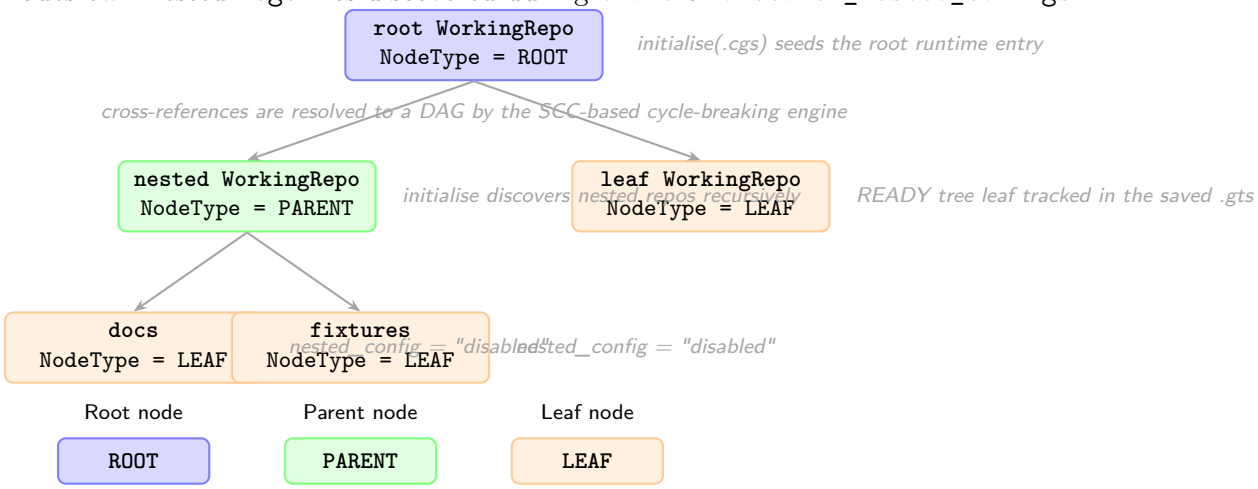
Registry entries in `WorkingGitTree` are indexed by a colon-separated repo ID derived from their position in the tree (e.g. `root:deps/child-repo:docs`). Cross-referenced parents therefore create a tangle-like declaration layer, but the expanded runtime graph remains a DAG after the cycle-breaking engine has run.

Cycle Breaking Engine

`fix_circularities` is a two-phase engine that converts a potentially cyclic dependency graph into a valid DAG:

1. **SCC detection (Tarjan's algorithm).** A directed dependency graph is built from the registry (edges: parent absolute path → child absolute path). `find_strongly_connected_components` identifies groups of paths that form a cycle. For each non-trivial SCC, an *Anchor* path is selected using three heuristics: (1) most external incoming edges, (2) closest to project root, (3) smallest SHA-256 hash. Back-edge entries — registry entries resolving to the anchor's path but parented inside a non-anchor SCC node — are flagged `is_external_reference = True` and removed.

Figure 3.3: `WorkingGitTree` is a parent–leaf graph of `WorkingRepo` nodes. Root and parent nodes own nested `.cgs` files discovered during `clone` or `discover_nested_configs`.



2. **Hash-compatibility deduplication.** Residual path-duplicate entries are removed when their synchronisation state is compatible with the canonical entry.

The clone engine uses a **sync stack** (set of absolute paths already entered into the clone pipeline) to exclude entries whose path is already being processed, providing defence-in-depth against any residual back-references that escape the registry cleanup phase.

`topological_sort(registry)` returns entries in parent-first order (Kahn's BFS algorithm) for safe sequential clone/pull operations. Runtime `pull` uses the same three-level direction: `ROOT -> PARENT -> LEAF`; the project root repository is pulled before child submodules are updated from their parent repositories.

Figure 3.6 shows the full object model for Tier 1.

3.4 Tier 2 — Actions

Actions are operations that read or mutate the Core tier. **Every action is gated by the current `TreeLifecycleState`**: actions that require a `READY` tree will refuse to execute and log a gating-refusal event if the tree is not ready.

Figure 3.7 shows the full lifecycle state machine. The public CLI presents this lifecycle through `initialise`, `pull`, and the READY-state sync commands; lower-level load/expand/-validate stages are internal steps of those workflows.

Action	Minimum state	Produces
`initialise(.cgs)`	none	clone tree + **READY**
`clean-init(.cgs)`	none	purge generated state + clone tree + **READY**
`purge(.cgs)`	none	removed generated root-level clone state
`initialise(.gts)`	none	loaded/restored **READY**
`pull(.cgs|.gts)`	READY	resynchronised **READY**
`checkout(.gts)`	READY	**READY**
`add`	**READY only**	READY
`commit <msg>`	**READY only**	READY
`push`	**READY only**	READY + updated hash
`tag <name>`	**READY only**	READY + updated tag
`freeze`	**READY only**	next `.gts` id

For `.cgs` refs, repositories can be declared by branch or by tag. Format validation is static and does not resolve either reference remotely. The runtime layer selects the declared target (tag takes precedence when both are present) and checks its availability through `GitRunner`.

The authoring boundary is explicitly `PARSE -> NORMALIZE -> VALIDATE`. TOML parsing produces authoring data; normalization expands `provider:owner/repository` identifiers and fills `main` branch defaults, `ssh`, automatic nested discovery, and relative paths. Validation receives only the complete canonical `CgsDocument`. Authoring syntax is therefore not the internal representation.

File and CLI authoring converge at this boundary. The CLI collects `-project` and repeatable `-repo` values, then calls the non-interactive `ComplexGitSyncClient.configure()` Python API. Python callers can use the same method directly. The facade delegates to `CgsDocument`; neither it nor the CLI owns repository grammar or normalization. Loading an equivalent `.cgs` file and using either definition path produce semantically identical canonical documents. `create-cgs` runs the same offline pipeline and serializes the result without entering Git runtime.

The boundary is also responsible for the reverse projection. `CgsDocument.to_git_tree()` creates the reference model and `CgsDocument.from_git_tree()` converts a reference or working tree back to canonical configuration. `GitTree.to_cgs()` remains only as a convenience delegate; it does not assemble repository tables or format TOML. The minimal serializer then removes deterministically reconstructible defaults. After the generated TOML is parsed and normalized again, its canonical representation must equal the one before serialization. Byte equality is not part of this contract.

Repository placement is parent-relative. At the top level the parent path is the project root; during nested discovery it is the repository whose `.cgs` is being expanded. Thus `../sibling` resolves to an existing sibling directory. Discovery compares normalized absolute paths and keeps an existing registry entry instead of inserting a duplicate.

Placement and traversal are independent. `nested_config = auto` searches the repository root for exactly one `*.cgs`, `disabled` stops discovery through that reference, and a relative `.cgs` path selects an exact file inside the repository. Explicit paths may not escape the repository root.

The textual `provider:owner/repository` grammar belongs exclusively to `cgs_format.py`. Its `parse_repo_id()` function owns syntax validation and splitting. Normalization uses that function, and CLI repository arguments must reuse it rather than introduce another parser.

The complete `.cgs` format pipeline is deterministic and offline. Parsing, normalization, validation, and serialization do not probe repositories or execute Git. Remote existence and branch/tag availability are resolved only after the validated document enters the runtime layer, through explicit `GitRunner` operations.

Provider behavior remains in `git_repo.py`: the `GitProvider` enum, canonical provider set, known-host map, and `RepoAddress` URL composer operate on already-separated identity fields. GitHub, GitLab, and Codeberg map respectively to `github.com`, `gitlab.com`, and `codeberg.org`. `custom` has no known-host entry and requires an explicit `gitprovider_url`; no fallback host is guessed.

Before any mutating workflow step (`commit`, `push`, `tag`, or `freeze`), the action layer now runs a workspace preflight validator. It checks dirty worktrees, detached HEADs, missing remotes, branch divergence, unresolved merges, missing submodule links, and stale recorded `commit_sha` values. Diagnostics are severity-based: warnings report actionable-but-allowed states, while blocking errors stop the operation. `tag` still requires a clean worktree and a non-existing tag.

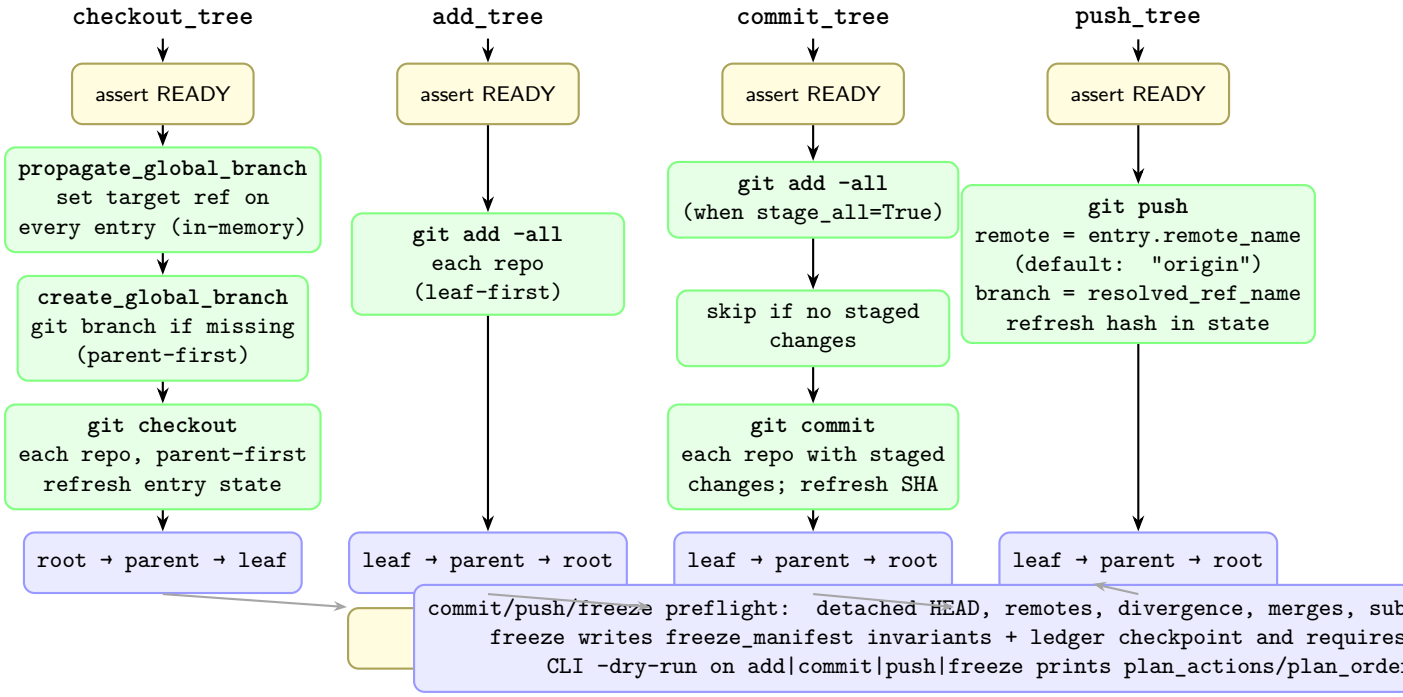
Formal `.gts` snapshot specification (T31)

`.gts` now carries explicit schema and deterministic hash metadata in the `[document]` table: `schema_version = "1.1"`, `hash_algorithm = "sha256"`, and `snapshot_hash = "<64-hex>"`.

The canonical digest is computed from a deterministic payload made of `project`, `tree_state`, and sorted `repo_state` records, excluding volatile generation metadata (`generated_at` and `command_origin`). Non-root entries require `parent_absolute_path`; READY repositories require `commit_sha`. This makes `.gts` a stable checkpoint primitive for repeatable workspace state comparison.

Figure 3.4 shows the internal sequence of the three tree-wide synchronisation actions implemented in `operations.py`.

Figure 3.4: Synchronisation operations: `pull/checkout_tree` process `ROOT -> PARENT -> LEAF`; `add_tree`, `commit_tree`, and `push_tree` iterate `LEAF -> PARENT -> ROOT` so that inner repos are synchronized before their parents.



3.5 Tier 3 — Client / API

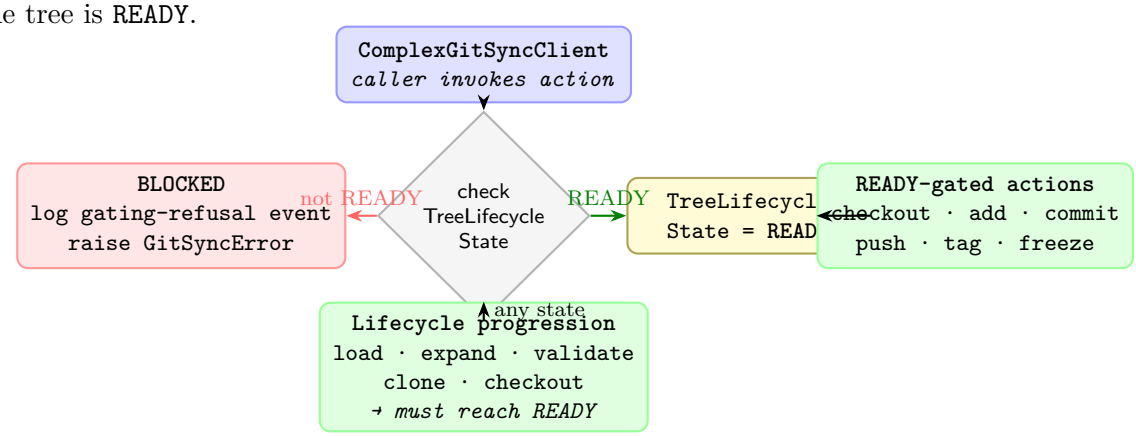
`ComplexGitSyncClient` is the single public facade. It:

- holds references to `Orchestre`, `GitRunner`, `RuntimeStateStore`, and the live `WorkingGitTree`;
- computes the current `TreeLifecycleState` on demand and gates every action against it;
- emits structured log events for every state transition, action start/end, fallback decision, and `.gts` write/load.

`Orchestre` is the coordination layer between the Client and the tree models. The CLI (`cli.py`) collects terminal arguments or interactive prompt values, then delegates to the reusable Client API. The Client delegates format semantics to `cgs_format.py`; runtime operations remain separate.

Figure 3.5 shows how the Client gates action access based on the live tree state.

Figure 3.5: Client state-gating: the Client reads `TreeLifecycleState` before delegating to an action. READY-gated actions (`checkout`, `add`, `commit`, `push`, `tag`, `freeze`) are blocked unless the tree is `READY`.



3.6 Module Layout by Tier

Listing 3.1: Source modules grouped by architectural tier

```
src/ComplexGitSync/
  # --- Tier 1: Core Data (centred on GitTree / GitRepo) ---
  git_repo.py                    GitRepo, WorkingRepo, RepoAddress,
                                 RepoNode, enums
  git_tree.py                    GitTree, WorkingGitTree,
                                 ProjectTreeState, tree utilities

  # --- Tier 2: Actions ---
  operations.py                  propagate_global_branch,
     create_global_branch
                                 checkout_tree, commit_tree, push_tree
  orchestre.py                   discover_nested_configs(),
                                 build_registry_from_cgs_document, render
                                    helpers

  # --- Tier 3: Client / API ---
  orchestre.py                   Orchestre, ComplexGitSyncClient
  cli.py                         build_parser / main
  orchestre.py                   GitRunner, RuntimeStateStore,
                                 CommandRunLogger + create_run_logger

  # --- Document layer (cross-cutting) ---
  config_document.py             ConfigDocument (format-neutral base)
  cgs_format.py                  CgsDocument (.cgs parse/normalize/
     validate/serialize)
  orchestre.py                   GtsDocument, GocDocument
  errors.py                      exception hierarchy
```

3.7 Document Hierarchy

Figure 3.8 shows the document class hierarchy.

All persistent documents share the `ConfigDocument` base class, which provides format-agnostic serialisation (`to_toml`, `to_json`, `to_yaml`) and factory methods (`from_toml`, `from_json`, `from_yaml`). Subclasses add document-specific validation and convenience properties. The base

is defined in `config_document.py`; `cgs_format.py` owns the complete `.cgs`-specific boundary, while `orchestre.py` consumes validated documents and retains runtime/orchestration responsibilities.

- **CgsDocument** (`.cgs`) — the authoring spec; describes the static project topology. *Never* a runtime snapshot.
- **GtsDocument** (`.gts`) — a generated snapshot capturing the exact state of the tree including commit SHAs and absolute paths. *Never* hand-edited.
- **GocDocument** (`.goc`) — a command script encoding a sequence of `cgitsync` commands with session defaults.
- **Local Git Register** (`.lgr`) — the project-local register that assigns one stable local id to each generated `.gts` and tracks the current snapshot pointer.

3.8 Registry Model

Figure 3.9 shows the registry model.

`WorkingGitTree` is the authoritative in-memory graph. It maps *repo IDs* (colon-separated path strings such as `root:deps/child-repo`) to `WorkingRepo` instances. `WorkingRepo` holds both the static identity declared in `.cgs` and the dynamic state updated during operations.

Builder functions in `orchestre.py` translate document objects into a live working tree:

```
# Load from authoring spec
registry = build_registry_from_cgs_document(document, config_path)

# Load from snapshot
registry = build_registry_from_gts_document(snapshot_document)

# Serialise to snapshot
gts_doc = build_gts_document_from_registry(
    registry, command_origin="clone", source_cgs_path=path
)
```

Figure 3.6: Core object model: `ComplexGitSyncClient` orchestrates through `Orchestre`, which owns the central `GitTree` composed of `GitRepo` instances. `RepoAddress` derives the remote URL from a `GitRepo`.

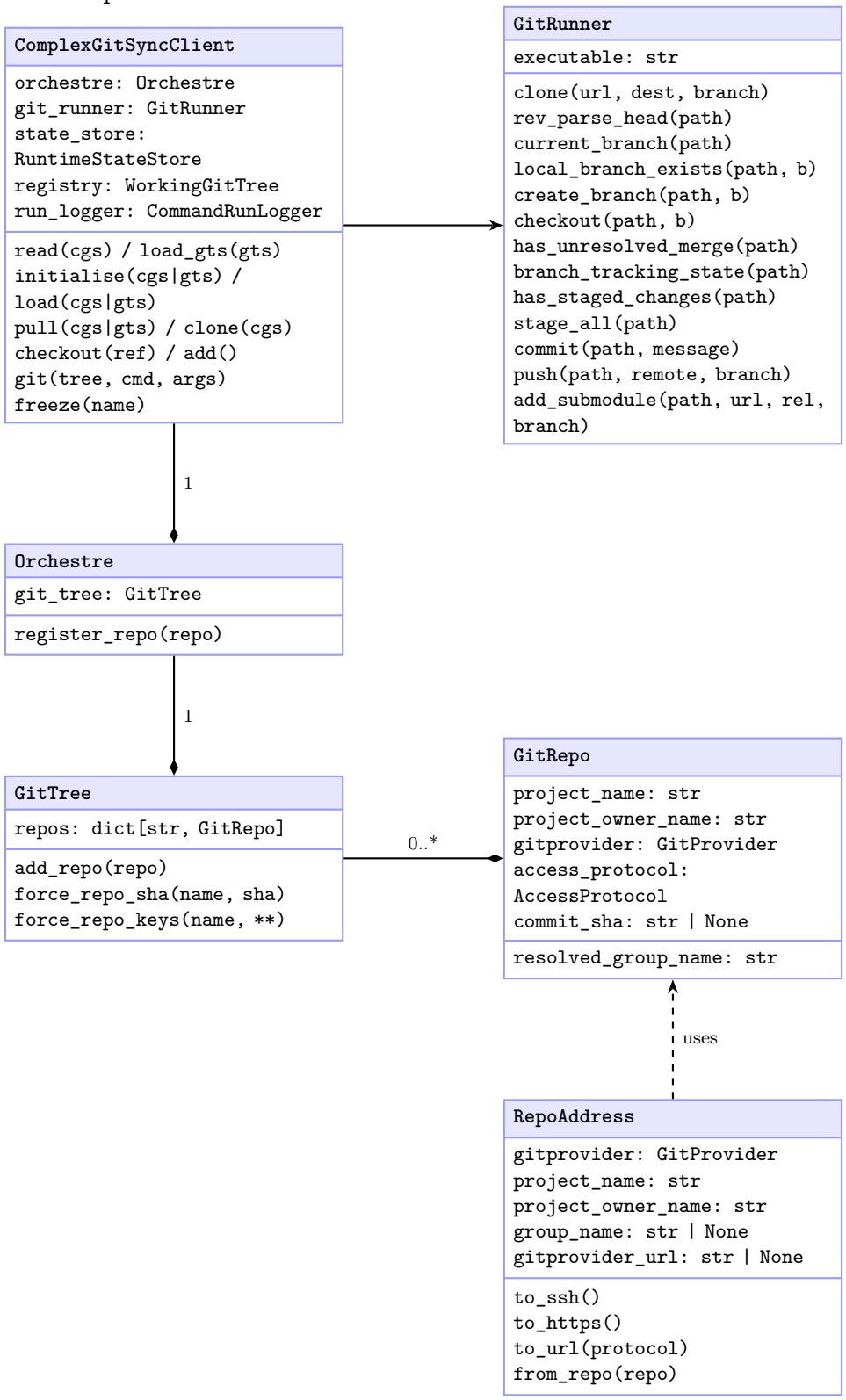


Figure 3.7: GitTree lifecycle state machine. The simplified user-facing lifecycle is `initialise(.cgs)` → clone → `READY`; `initialise(.gts)` → restore → `READY`; `pull(.cgs|.gts)` → resync → `READY`. Synchronized Git operations (`checkout`, `add`, `git("commit")`, `git("push")`, `git("tag")`, `freeze`) are gated on `READY`.

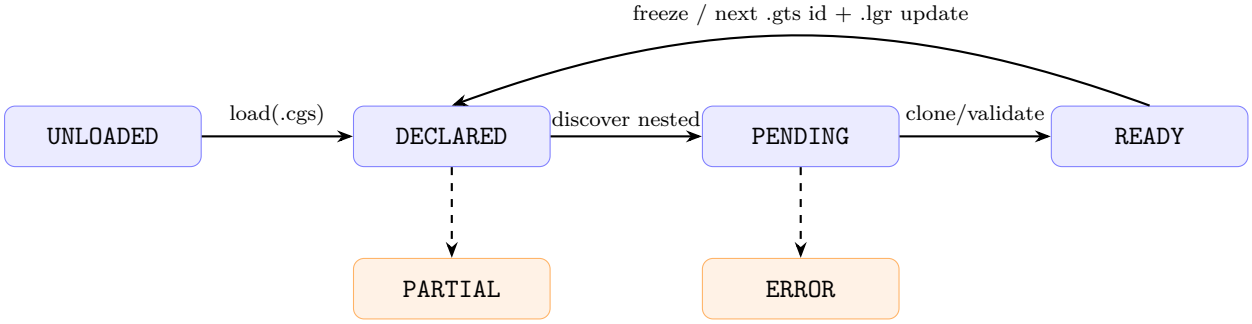


Figure 3.8: Document class hierarchy. `ConfigDocument` is the abstract base; each concrete subclass parses and validates one file format.

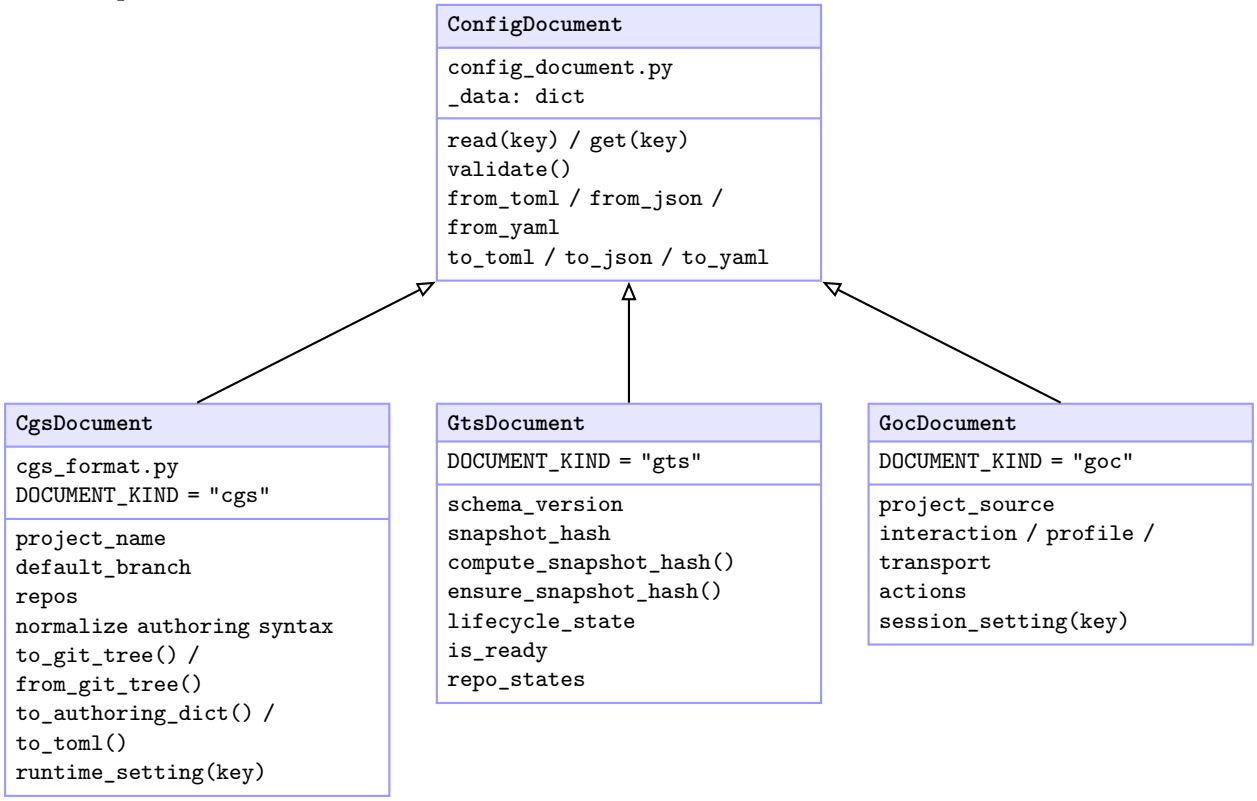
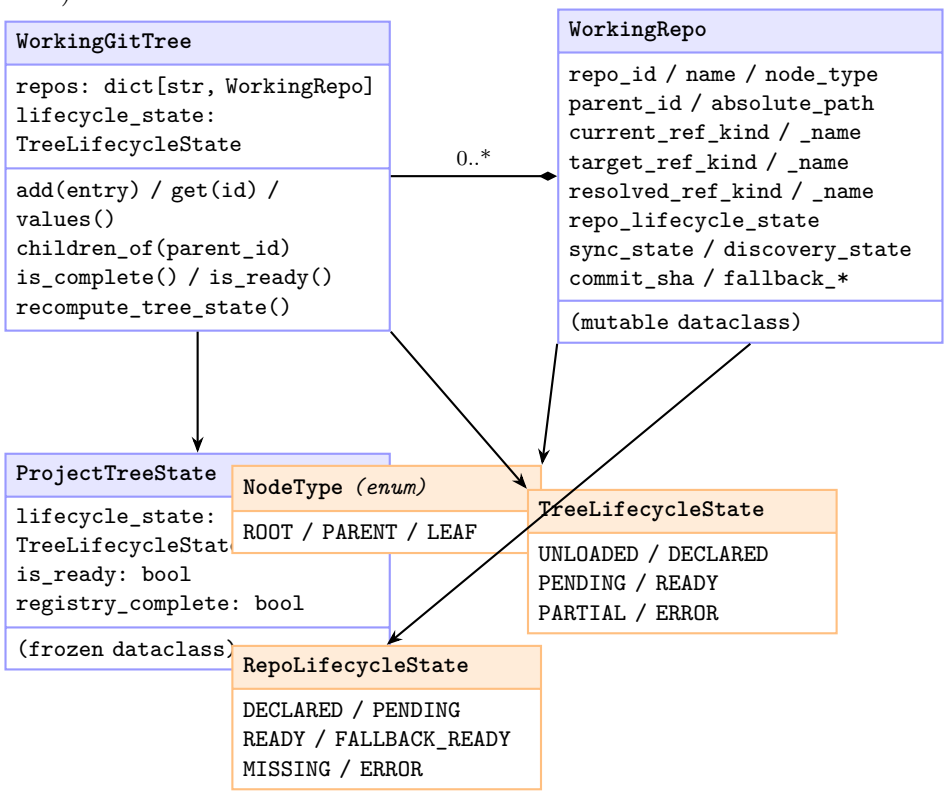


Figure 3.9: Registry model. `WorkingGitTree` is the authoritative in-memory graph; `WorkingRepo` records both static identity (from `.cgs`) and dynamic state (from synchronisation operations).



Chapter 4

User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by `ComplexGitSync`.

4.1 CLI Overview

The single entry-point is the `cgitsync` command.

```
$ cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

4.2 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone all repos → `READY` *(new project)*
   `initialise(.gts)` → restore from snapshot → `READY` *(existing project)*
2. `pull(.cgs|.gts)` → resync an existing tree
3. `checkout(.gts)` / `add` / `commit` / `push` / `tag` → git operations on the tree
4. `freeze` → emit the next `.gts` id and update `<Project_name>.lgr`

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

4.3 Commands

The CLI exposes two levels once a `GitTree` is `READY`. The minimalist level is `initialise`, `clean-init`, `freeze-release`, `freeze-release-force`, `status`, `view-tree`, and `launch-release`. The expert level exposes the individual operations: `branch`, `checkout`, `purge`, `validate`, `clone`, `pull`, `pull-force`, `add`, `commit`, `push`, `tag`, and `freeze`. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

4.3.1 initialise

```
$ cgitsync initialise <spec.cgs> [--output-path DIR]
$ cgitsync initialise <snapshot.gts>
$ cgitsync initialise --project NAME --repo PROVIDER:OWNER/REPO [--repo
   ...] [--output-path DIR]
```

Unified initialisation entry point. Dispatches on file extension:

- `.cgs` source (clone mode): clones the full repository tree. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...` On success, ends in `READY`. If clone setup fails because stale generated state is still present, the CLI prints `Try clean-init method`.
- `.gts` source (restore mode): loads a saved snapshot directly. On success, ends in the lifecycle state recorded in the snapshot.
- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()`; that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a `READY` tree or fail explicitly.

4.3.2 create-cgs

```
$ cgitsync create-cgs --project CGSil1 \
    --repo github:flipoyo/ComplexGitSync \
    --repo codeberg:GX4G/GX4G \
    --output CGSil1.cgs
```

Builds the canonical `CgsDocument` through the offline format pipeline and serializes concise `.cgs` TOML. It performs no Git or network work.

4.3.3 clean-init

```
$ cgitsync clean-init <spec.cgs> [--output-path DIR]
```

Recovery-oriented initialisation for a `.cgs` source. It follows the same high-level pipeline as `initialise(.cgs)`, but inserts a cleanup phase before cloning:

```
load->expand->validate->purge->clone
```

Use it when an earlier initialisation attempt left partial child checkouts, stale submodule metadata, or an obsolete root `.gitmodules` file.

4.3.4 purge

```
$ cgitsync purge <spec.cgs> [--output-path DIR]
```

Removes generated clone state from the resolved `CGSHOME` without cloning. It deletes child repository directories declared directly under the project root, project-local `*.lgr` files, and the root `.gitmodules` file. Nested directories that are not immediate root children are left untouched by this command.

4.3.5 pull

```
$ cgitsync pull [spec.cgs|snapshot.gts]
```

Resynchronises the project tree. `.cgs` sources reload the source, discover nested configs, then resynchronise the loaded tree. `.gts` sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: `root -> parent -> leaf`. The root repository receives `git pull -ff-only`; each child repository is then updated through its parent submodule link. Use `launch-release` when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI prints `You can try cgitsync pull-force command`.

4.3.6 pull-force

```
$ cgitsync pull-force [spec.cgs|snapshot.gts]
```

Destructively resynchronises the same tree. The root repository is forced to the selected remote branch with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`; child submodules are then force-updated parent-first. This command can discard local uncommitted changes and untracked files.

4.3.7 clone

```
$ cgitsync clone <spec.cgs> [--target-dir DIR] [--output-path DIR]
```

Clones the full repository tree declared by the `.cgs` file. This CLI command delegates to `ComplexGitSyncClient.clone`. When `-target-dir` is omitted, the project root is derived from the specification. `-output-path` provides the parent directory used to derive the project root.

For each repository, `cgitsync` tries the repo target branch first and falls back to the repo fallback branch when the preferred branch does not exist on the remote. Nested `.cgs` files are discovered after parent repositories are cloned, so explicit and automatic nested trees are materialised during the same command.

4.3.8 checkout

```
$ cgitsync checkout <branch> [--gts <snapshot.gts>] [--ref-kind branch|
   tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the READY registry from `-gts`, then propagates the ref, creates missing local branches, and runs `git checkout` in every repo. On success the tree remains `READY` and a new `.gts` snapshot is written.

4.3.9 add

```
$ cgitsync add [--gts <snapshot.gts>] [--dry-run]
```

Stages all changes across the tree (`git add -all`), leaf-first. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitsync` prints the leaf-first execution plan and does not mutate repositories.

4.3.10 commit

```
$ cgitsync commit <message> [--gts <snapshot.gts>] [--no-stage] [--dry-
    run]
```

Commits changes across the tree, leaf-first. Loads the READY registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a `READY` tree; the tree remains `READY` on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, stale recorded `commit_sha` values, or child repositories that are not linked as submodules. With `-dry-run`, `cgitsync` prints the planned stage/commit sequence without performing git writes.

The commit message conventionally ends with `CGS#VERSION`.

4.3.11 push

```
$ cgitsync push [--gts <snapshot.gts>] [--dry-run]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the READY registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to `"origin"`) on its currently resolved branch. Refreshes the stored hash in `GitTree`. Requires a `READY` tree; the tree remains `READY` on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, behind/diverged branches, and missing submodule links; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgitsync` prints the push plan without mutating any repository.

4.3.12 freeze-release

```
$ cgitsync freeze-release <name> <message> [--gts <snapshot.gts>] [--dry
    -run]
$ cgitsync freeze-release-force <name> <message> [--gts <snapshot.gts>]
    [--dry-run]
```

Minimalist release workflow from a READY tree. It chains public operations in order: `add -> commit -> pull -> push -> freeze`. The force variant uses `pull-force` instead of `pull`. The release name becomes the shared tag and freeze snapshot label; the message is used for the release commit.

4.3.13 freeze

```
$ cgitsync freeze <name> [--gts <snapshot.gts>] [--dry-run]
```

Performs a release freeze across all repos, leaf-first: stage/commit, create+push the shared release tag, refresh SHA/state, and write a `.gts` snapshot. Loads the READY registry from `-gts`. Requires `READY` and leaves the tree `READY`. Freeze preflight checks remote availability, branch alignment, detached HEADs, unresolved merges, tag absence, and submodule links for all parent/child repository boundaries. Dirty worktrees and stale recorded `commit_sha` values are reported as warnings because freeze will stage, commit, and snapshot them. Freeze snapshots also include a deterministic `freeze_manifest` section that records freeze invariants (immutable snapshot, validated workspace, synchronized tag, `release-name`, ledger checkpoint) and declares `launch_state` as the canonical restore operation. Their immutable snapshot filename includes the release, for example `gts-000001-release-2026.05.gts`, while the ledger id remains `gts-000001`. With `-dry-run`, `cgitsync` prints the planned add/commit/tag/push graph for the loaded workspace without mutating it.

4.3.14 launch-release

```
$ cgitsync launch-release <name> [--gts <snapshot.gts>]
```

Checks out the frozen release tag `<name>` across the loaded READY tree, parent-first, and writes a fresh `.gts` snapshot. This is the release oriented CLI path for returning a workspace to a frozen version; direct tag creation remains available from the Python client API as `tag()`.

The integration suite validates local no-network lifecycle coverage through the CLI path, including initialisation and the READY git command cycle `add -> commit -> push -> freeze -> launch-release`.

4.3.15 view-tree

```
$ cgitsync view-tree [spec.cgs|snapshot.gts] [--depth N] [--collapse
   REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state: `name [branch] local_state sync_state`. Use `-depth` to limit visible levels and repeat `-collapse` to hide selected subtrees.

Branch Topology Propagation Rules (T35):

1. **Reference branch**: The root repository's current branch is the canonical reference for all repos.
2. **Leaf-to-root inheritance**: Branch targeting flows root-first via `propagate_global_branch`. `validate-topology` verifies the on-disk state without issuing any git writes.
3. **Allowed divergence**: Repositories in a frozen (`TAG`) state produce a `tag_divergence` diagnostic that does *not* make the topology incoherent.
4. **Incoherent states** (blocking): `misaligned_branch` (different branch from root) and `detached_head` (detached HEAD without a tag reference).

4.3.16 status

```
$ cgitsync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

4.4 Document Formats

`.cgs` means **ComplexGitSync** specification, `.gts` means **GitTreeState** snapshot, `.goc` means **GitOrchestrationCommand** plan, and `.lgr` means **Local Git Register**.

4.4.1 Local Authoring Spec (.cgs)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
```

```
  "github:flipoyo/CGSih1",
]
```

Repository identifiers have the form `provider:owner/repository`. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

`PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument`

The canonical providers and hosts are:

Provider	Host	Remote generation
`github`	`github.com`	SSH and HTTPS
`gitlab`	`gitlab.com`	SSH and HTTPS
`codeberg`	`codeberg.org`	SSH and HTTPS
`custom`	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.
    example.com" }
```

No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.gi` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main`, makes `fallback_branch` equal to that default, selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `..`

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
     nested_config = "disabled" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

4.4.2 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- `auto` (default) loads the sole root-level `*.cgs` file, if one exists; multiple matches are rejected as ambiguous.
- `disabled` does not inspect the repository for nested config.
- A relative path ending in `.cgs` loads that exact file inside the repository. It may not escape the repository root.

For the integration topology, `CGSil2.cgs` contains:

```
{ repository = "github:flipoyo/CGSih1", relative_path = "../CGSih1",
   nested_config = "disabled" }
```

Because the config describes the tree below `CGSil2`, the path resolves from that directory:

```
<CGSHOME>/CGSil2/../CGSih1 -> <CGSHOME>/CGSih1
```

It therefore references the existing sibling declared by `CGSil1.cgs`, instead of cloning another `CGSih1` under `CGSil2`. Discovery recognizes the already-registered absolute path and keeps the canonical entry. The cross-reference is `disabled` because the canonical root-level `CGSih1` entry is responsible for loading `CGSih1.cgs`.

4.4.3 Git Tree State Snapshot (`.gts`)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.

- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.
- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, ComplexGitSync writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.
- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

4.4.4 Planning / GOC Documents (`.goc`)

Used for project identity and planning metadata. Supported in TOML, YAML, or JSON. YAML support requires the optional `PyYAML` dependency.

4.4.5 Local Git Register and Sync Ledger (`.lgr`)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id         = "lgr-000001"
parent_sync_ids = []
operation       = "clone"
timestamp       = "2026-05-20T19:48:50.159Z"
actor           = "username"
workspace_hash  = "5f7c8a2d..."   % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos  = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

4.5 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..`; later commands can also walk upward from the current directory to find `.cgitsync/state/`. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

4.6 Error Reference

`ConfigValidationError` Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

`GitSyncError` Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

4.7 Troubleshooting

"`gitprovider_url` is required for custom provider" Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

`validate` exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a `repository` shorthand, include `project_owner_name` and `project_name`.

`Clone destination already exists and is not empty` Re-run `clone` with `-target-dir` to force a clean location, or let `cgitsync` choose a suffixed default directory.

`No cloneable branch found for <repo>` Confirm that the remote exposes either the repo's `default_branch` or its `fallback_branch`.

PyYAML import error when using `.goc` files Add PyYAML in the Pixi environment: `pixi add pyyaml`.